# Interactive Finite Automata Generator: Regular Expression to NFA and DFA

Department of CSE
COURSE: COMPILER DESIGN –(24CS2235F)
SECTION-5

Review-1

Team-12

Under the Guidance of: P.Krishna Kishore

Team Members
2420030046-N.Lasya
2420030471-N.Nikitha Reddy
2420030119-C.Akshara



# Introduction

- Regular Expressions and Finite Automata are fundamental concepts in **Compiler Design and Formal Language Theory**. Regular Expressions provide a concise way to describe patterns, while Finite Automata provide a computational model for recognizing those patterns. The conversion of a Regular Expression into an NFA and subsequently into a DFA is an important process used in areas such as lexical analysis and pattern recognition.
- However, manually performing these conversions can be difficult to understand because they involve multiple states, transitions, epsilon transitions, epsilon closures, and state combinations.
- The proposed project, **"Interactive Finite Automata Generator: Regular Expression to NFA and DFA,"** is a web-based application that automates this conversion process. The user can enter a valid Regular Expression, and the system dynamically generates its corresponding NFA using **Thompson's Construction** and converts the NFA into a DFA using **Subset Construction**.
- The application provides transition tables and interactive graphical visualizations of both NFA and DFA, allowing users to understand the conversion process clearly and interactively.

# Abstract


- The project **“Interactive Finite Automata Generator: Regular Expression to NFA and DFA”** is a web-based application developed to simplify the understanding of Regular Expressions and Finite Automata concepts in Compiler Design. The system allows users to enter a Regular Expression through an interactive user interface and automatically generates the corresponding NFA (Non-Deterministic Finite Automaton) and DFA (Deterministic Finite Automaton).
- The system first validates the given Regular Expression and converts it into postfix notation. It then uses **Thompson’s Construction** to generate the NFA and **Subset Construction** to convert the NFA into a DFA. The generated automata are presented through transition tables and interactive graphical visualizations, including start states, accepting states, transitions, and epsilon transitions.
- The application is developed using **React and Vite for the frontend** and **Python Flask for the backend**. The system supports common Regular Expression operators such as union (|), concatenation, Kleene star (*), plus (+), optional (?), and parentheses. Since the system is dynamic, users can enter different Regular Expressions without modifying the program. The project provides a practical learning platform for understanding **Compiler Design, Regular Expressions, NFA, DFA, Thompson’s Construction, and Subset Construction**.

# Objectives

The main objectives of the project are:

- To develop an **interactive web-based tool** for converting Regular Expressions into finite automata.
- To automatically generate an **NFA using Thompson's Construction**.
- To convert the generated NFA into a **DFA using Subset Construction**.
- To provide **dynamic Regular Expression input** without requiring changes to the source code.
- To display NFA and DFA **transition tables** clearly.
- To provide interactive **graphical visualizations** of NFA and DFA states and transitions.
- To implement **Regular Expression validation** and provide meaningful error messages for invalid expressions.
- To help students understand important concepts such as **epsilon transitions, epsilon closure, state transitions, and DFA state formation**.
- To reduce the complexity of manually performing Regular Expression to NFA and DFA conversions.
- To provide an easy-to-use educational tool for **Compiler Design and Formal Languages**.

# Proposed Methodology

- The proposed system follows a sequential conversion methodology:

**Step 1: Regular Expression Input**

- The user enters a Regular Expression through the web interface.
- Example:
- (a|b)*abb
- The system supports operators such as:
- | Union
- . Concatenation
- * Kleene Star
- + One or More
- ? Optional
- () Grouping

**Step 2: Regular Expression Validation**

- The backend checks whether the entered Regular Expression is syntactically valid.
- It detects errors such as:
- Empty input
- Mismatched parentheses
- Invalid operators
- Incorrect operator placement
- If the expression is invalid, the system displays an appropriate error message.

**Step 3: Infix to Postfix Conversion**

The validated Regular Expression is converted from infix notation into postfix notation.

For example:

(a|b)*abb

is converted into an equivalent postfix expression.

This makes it easier to process the expression algorithmically.

**Step 4: Thompson's Construction**

The postfix Regular Expression is processed using **Thompson's Construction**.

The system creates NFA states and transitions for:

- Symbols
- Union
- Concatenation
- Kleene Star
- Plus
- Optional

Epsilon (ε) transitions are used wherever required.

**Step 5: NFA Generation**

The resulting NFA contains:

- Start state
- Intermediate states
- Accepting state
- Symbol transitions
- Epsilon transitions

The NFA transition table and graphical representation are then generated.

**Step 6: Epsilon Closure**

For NFA-to-DFA conversion, the system calculates the **epsilon closure** of NFA states.

The epsilon closure identifies all states reachable through epsilon transitions without consuming an input symbol.

**Step 7: Subset Construction**

The system uses **Subset Construction** to convert the NFA into a DFA.

Each DFA state represents a set of NFA states.

For example:

A = {q0, q1, q3}

B = {q2, q4}

C = {q5}

The process continues until all possible DFA states and transitions are generated.

**Step 8: DFA Generation**

The resulting DFA contains:

- DFA start state
- DFA states
- DFA transitions
- DFA accepting states

The DFA transition table and graphical representation are displayed.

**Step 9: Visualization**

The frontend displays the generated NFA and DFA as interactive graphs.

Users can:

- View states
- View transitions
- Identify start states
- Identify accepting states
- Zoom
- Pan
- Examine the automata structure

# System Architecture

